2026 Cornell Trading Competition Case Packet

Last Updated: 10/14/2026

Welcome to the 2026 Cornell Trading Competition! We are excited to host you all on Roosevelt Island in New York City for a fun day of trading games and networking!

This case packet contains detailed specifications about the cases and games that will be played during the competition. **Please read the packet in full before the competition. The systematic equities also requires prior coding work, including performing data analysis and writing a trading bot.**

We hope you find the competition fun and educational!

General Logistics

Location:
Verizon Executive Education Center, on Roosevelt Island in New York City
Address: 34 N Loop Rd, New York, NY 10044
Once in NYC, we recommend taking the aerial tram or subway (F train) to reach Roosevelt Island.

Time:
Check-in will be from **8 am to 8:30 am**. We encourage you to arrive earlier if you want time to eat breakfast, which we will provide at the venue. The competition ends at **6:30 pm**. You must stay for the full duration of the competition to receive travel reimbursements. Please let us know beforehand if you have extenuating circumstances and must leave early or arrive late.

Important Links:
Schedule (tentative)
Slack Channel

Prize Breakdown:
Prizes will be awarded to the top 3 teams in each of our 3 cases, as well as the top 3 overall teams. We will update this packet with the full prize structure once it is finalized. Currently, we plan on giving away a total of $12,000 in monetary prizes. In the overall scoring, your rank in

each case is weighted as follows: Mario Markets (20%), Derivatives (35%), Systematic Equities (45%).

Note: Events hosted by our gold sponsors will not contribute to the overall competition scoring. For those events, prizes will be provided by the gold sponsor and not the overall competition prize pool.

Sponsors:
Thank you to our sponsors for their generous support of the competition. Many of them will also be attending the career fair, speaking to students, and giving merch!

Gold Sponsors: Susquehanna, Jump Trading
Silver Sponsors: DRW, IMC, Jane Street, Hudson River Trading, Citadel | Citadel Securities, JPMorganChase Markets
Bronze Sponsors: Old Mission, Optiver, Walleye Capital, D.E. Shaw & Co, Millennium, Five Rings, Xantium

TODOs:
Before the competition, you should work with your team and:

- **Read through all case packets and familiarize yourself with trading terminology.** Think about the contracts and market behavior, and formulate a trading plan with your team.
- For the Systematic Equities case, you will need to perform data analysis on the sample data, and devise and test a trading algorithm. While you are able to trade manually, most of the trading should be done by your bot (there are a lot of symbols and a lot of data – too much for any human to trade!). **This case will require the most prior preparation, please start early.**
    - Generative AI is allowed. We recommend using it responsibly: you should still understand your code, and make design/strategy decisions yourself.

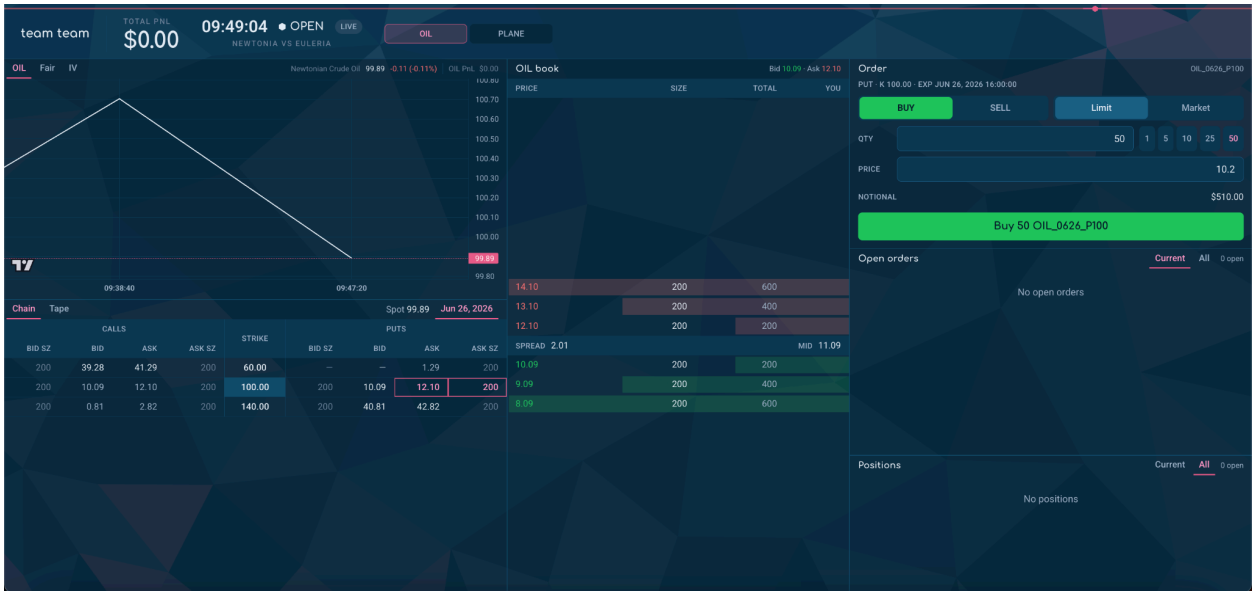
If you have questions, please send them in the #questions channel on Slack.

The case packets can be found in separate "Document Tabs" in the outline on the left-hand side.

Trading UI

Below is a sample of our exchange's UI, on which you will be trading. The screenshot below is the UI for the derivatives case (hence the options in the bottom left), but the UI will be roughly the same for all cases. Products are shown at the top, the order book is in the middle, and you can place orders on the right side.

Prior to the competition, we recommend familiarizing yourself with the UI. Make sure you understand what each tab, button, and piece of information means!



If you are unfamiliar with how an order book works, a **bid** is a price that someone is willing to buy at. You can see the current bids in green. An **offer/ask** is a price that someone is willing to sell at. They are in red. The **size/quantity** is the number of contracts quoted.

Mario Markets

Prediction Markets Case Packet

1. Case Overview

In Mario Markets, your team will trade prediction contracts tied to a live Mario Kart competition. Four human racers and eight computer-controlled (CPU) racers compete in each race. As results are revealed, you will update your expectations for future races and series outcomes, then decide which contracts to trade. Your objective is to generate the highest total profit and loss (PnL).

Your Role:
You are a trading team on the Mario Markets exchange. You can trade contracts on the four human racers, including race winners, series winners, and the scores of the top two human racers. Successful trading depends on how you value these outcomes as the competition progresses, not on which racer you personally support.

Racers per race: 4 human + 8 CPU
Competition format: 2 series of 4 races (8 total)
Market types: Race winner; series winner; top-two series scores

2. Game Setup and Race Scoring

- Each series consists of four races, and tracks are randomly selected.
- The four human racers use visually distinct characters chosen from Daisy, Peach, Birdo, and Yoshi. These characters have identical underlying statistics.
- Every human racer uses the same kart configuration: Teddy Buggy, Roller Tires, and Cloud Glider.
- Points are awarded based on overall finishing position among all 12 racers, including the CPUs. Each human racer's series score is the sum of their points across that series's four races.

Points Awarded Per Race:

1st: 15 points	2nd: 12 points	3rd: 10 points	4th: 9 points
5th: 8 points	6th: 7 points	7th: 6 points	8th: 5 points
9th: 4 points	10th: 3 points	11th: 2 points	12th: 1 point

The CPUs affect how many points human racers earn, but only the human racers can win the prediction-market contracts described below.

3. Markets Available

All contracts refer to the four human racers. The exchange offers three types of markets, each with its own settlement rule and position limit.

A. Series Winner Markets | Binary (Position limit: 20)

Contracts: Four per series, one corresponding to each human racer.
Settlement: After the four races, the contract on the human racer with the highest cumulative series score settles to 100; the other contracts settle to 0.
Tie rule: If human racers tie for the highest cumulative series score, the contracts on the tied racers each settle to 50 instead of 100. Contracts on all other racers settle to 0.

B. Race Winner Markets | Binary (Position limit: 10)

Contracts: Four per race, one corresponding to each human racer.
Settlement: After the race, the contract on the highest-placing human racer settles to 100; the other three settle to 0.
Important: A human racer does not need to finish first overall to win this market. If a CPU finishes first overall and a human finishes second, that human wins the race-winner market.

C. Top-Two Series Score Markets | Non-Binary (Position limit: 100)

Contracts: Three per series, based on the final four-race cumulative scores of the human racers.
#1 Score: Cumulative score of the highest-scoring human racer.
#2 Score: Cumulative score of the second-highest-scoring human racer.
Top-2 Total: Sum of the #1 Score and #2 Score.
Settlement: These contracts settle to their actual point values, not to binary values.

Illustrative Settlement Examples:
If the four human racers finish a series with 49, 42, 34, and 28 points, the 49-point racer's series-winner contract settles to 100 and the others to 0. The #1 Score, #2 Score, and Top-2 Total contracts settle to 49, 42, and 91.
If two human racers instead tie for first in a series with 45 points each, both of their series-winner contracts settle to 50. The #1 Score and #2 Score contracts each settle to 45, and Top-2 Total settles to 90.

4. How the Case Runs

The competition has two four-race series. Each completed race determines its race-winner market and contributes points to that series's human-racer leaderboard. When the fourth race of a series finishes, the corresponding series-winner and top-two score contracts are settled.
Settlement schedule:

• After each race: That race's Race Winner contracts.
• After Race 4: Series 1 Winner and Series 1 Top-Two Score contracts.
• After Race 8: Series 2 Winner and Series 2 Top-Two Score contracts.

Trading Availability

You may trade any market that has not yet settled. This includes race-winner markets for upcoming races as well as any unresolved series-winner and top-two score markets. Once a race is complete, its race-winner contracts settle and are no longer available to trade.
For example, during Race 2, the Race 1 winner contracts have already settled. The markets for later races and all other unresolved series contracts remain available.

5. Key Terminology

- Binary contract: A contract settling to 100 or 0 based on an outcome. The series-winner tie rule provides an exception, with a settlement value of 50.
- Non-binary contract: A contract settling to a numerical result, here a cumulative series score or the sum of two scores.
- Series score: The total race points earned by a human racer over one four-race series.
- Settlement: The final value assigned to a contract when its race or series ends.
- Position limit: The stated limit on the size of a position in a market.
- PnL (profit and loss): The net trading profit or loss earned from positions and their settlement.

6. Scoring

Final rankings are determined solely by each team's total PnL over the case. The team with the highest final PnL wins.

Systematic Equities Case Packet

Click here for historical data and sample code!

1. Case overview

The world

The Archipelago has four islands:

- Cascadia Prime, in the north
- Coral Key and Azure Isle, in the east
- Palm Atoll, in the south-west

Ten companies and two ETFs trade on the Archipelago Exchange. The companies are in three groups:

- **Island businesses (5 companies)** own sites on the islands: a port, resorts, fish farms and fleets, the power grid and ferry terminals.
- **Rebuilders (3 companies)** repair the damage: a construction company, a materials supplier and a generator rental company.
- **Insurers (2 companies)** pay for the insured losses: a primary insurer and its reinsurer.

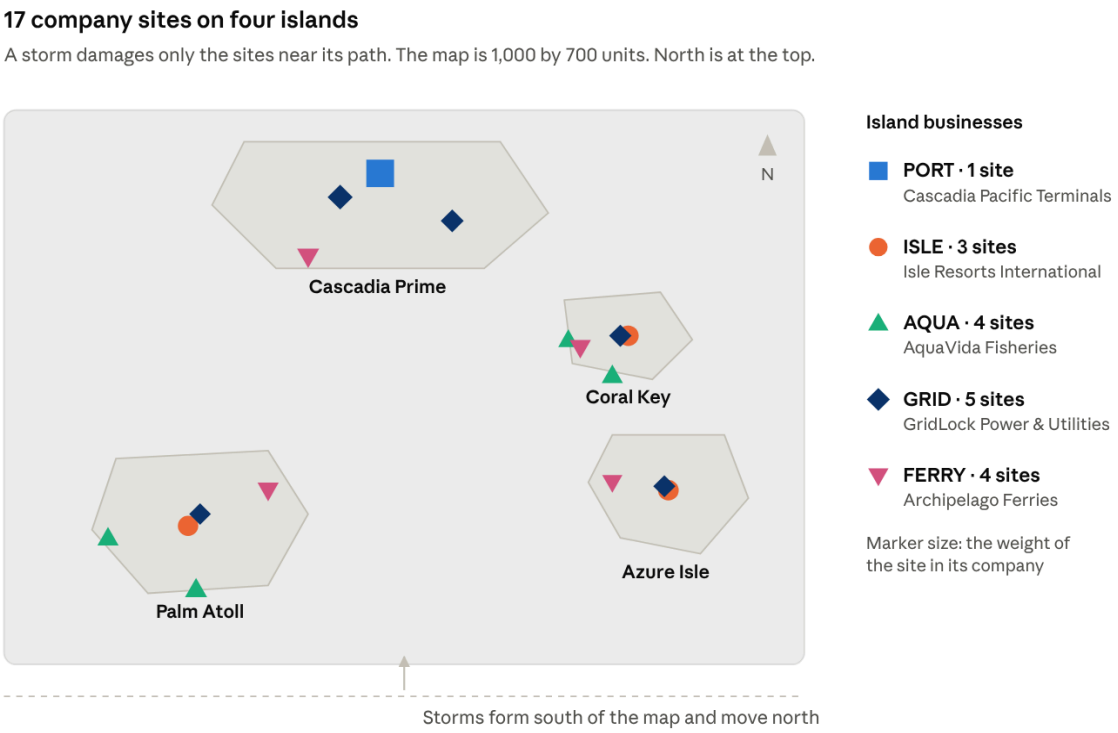


Figure 1. The 17 company sites. Marker shape shows the company; marker size shows how much of the company's value the site carries. Coordinates and weights are in `case.json`.

The season

The case is one hurricane season with three rounds. In each round, one hurricane forms south of the islands and moves north. A hurricane has these effects:

- It damages the sites near its path. The damage is permanent.
- It closes a site while the wind at that site is too strong.
- It gives work to the rebuilders.
- It causes insured losses that the insurers must pay.

Damage accumulates over the season: each storm adds to what the earlier storms did.

Nobody knows the path of a new storm in advance. Forecasters publish outlooks, rumors circulate on the wire, and prices react to both, not always correctly.

Your task

Your team trades the 10 stocks and the 2 ETFs on the Archipelago Exchange. You trade against the house and the other teams. You can trade by hand, with bots, or both. Most of the trading should be done algorithmically since there are lots of symbols and data!

At the end of round 3, each symbol settles at its fundamental value. That is the company's value after all of the season's damage. Your task is to find these values before the market does, and to trade on them.

2. How the case runs

The case has 3 rounds. Each round is one market day with one storm. Three things carry over from one round to the next:

- your positions
- the damage to each company
- the total insured losses of the season

Round	Market day	Events
Round 1	Day 1	Storm 1.
Round 2	Day 2	Storm 2. Each company starts from its condition after storm 1.
Round 3	Day 3	Storm 3. At the close, each symbol settles at its fundamental value.

Announcements can change the economics of a company at any point in the season.

The clock

The exchange uses a fast market clock. One market day, from 09:30 to 16:00, takes 13 real minutes. One real second is 30 market seconds. All feed timestamps use the market clock.

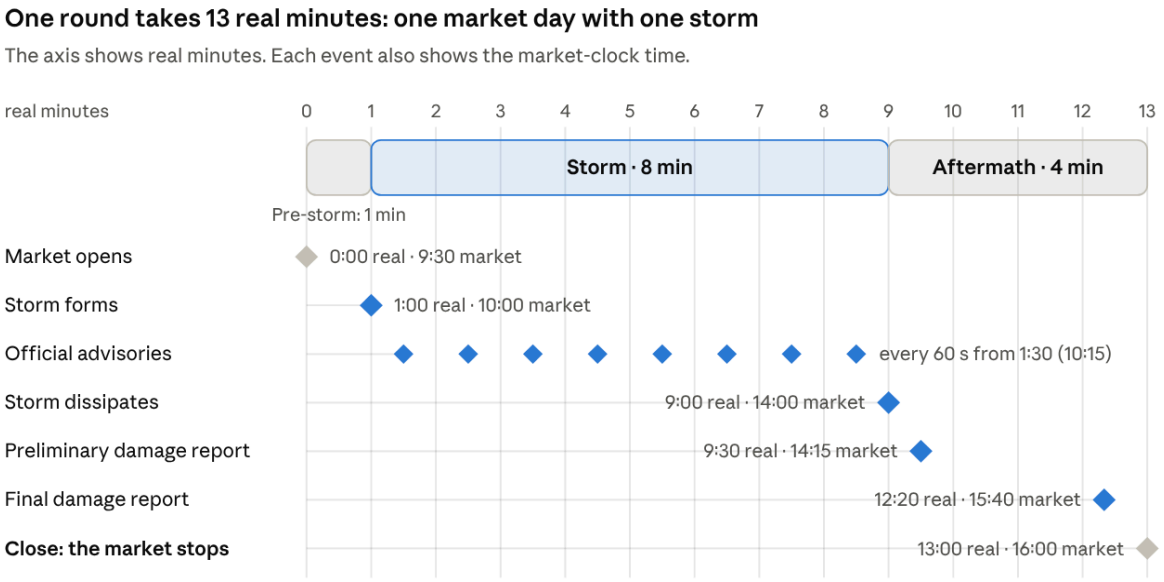


Figure 2. One round, drawn to scale. The axis is real time; each event is also labelled with its market-clock time.

3. The instruments

Twelve symbols trade: 10 stocks in three groups, and 2 ETFs. The starting price is the house price at the first open.

Ticker	Company	Group	Value depends on	Starting price	Position limit
PORT	Cascadia Pacific Terminals	Island business	1 port (Cascadia Prime)	72.48	200
ISLE	Isle Resorts International	Island business	3 resorts (Coral Key, Azure Isle, Palm Atoll)	50.26	300
AQUA	AquaVida Fisheries	Island business	2 farms, 2 fleets (Palm Atoll, Coral Key)	27.33	500
GRID	GridLock Power & Utilities	Island business	5 substations (all islands)	36.41	400
FERRY	Archipelago Ferries	Island business	4 terminals (all islands)	17.91	800
RECON	Reconex Construction	Rebuilder	Repair work	52.38	500
MAT	Materials Holdings	Rebuilder	Repair work	35.81	700
GEN	Backup Power Co.	Rebuilder	Power grid outages	40.30	600
HAVEN	HavenBridge Insurance	Insurer	Insured losses	39.55	250
REIN	Meridian Re	Insurer	Insured losses (reinsurance)	89.36	200
ARCH	Archipelago Index	ETF	1 PORT, 1 ISLE, 2 AQUA, 1 GRID, 2 FERRY, 1 HAVEN	289.20	50
REBLD	Rebuild & Recovery ETF	ETF	2 RECON, 2 MAT, 1 GEN	216.68	100

The file `case.json` gives the coordinates and the weight of each site. It also gives a description of each company. Read the descriptions closely; they contain details that affect value.

What changes the value of a company

- **Island businesses** lose value when a storm damages their sites. The damage is permanent. A closed site can also lose revenue.
- Rebuilders gain from damage to the island businesses, with diminishing returns: crews and supplies are limited, so each extra dollar of damage adds less value. Each rebuilder does a different type of work:
    - RECON repairs the port, the grid and the ferry terminals.
    - MAT supplies materials for all repairs.
    - GEN supplies power while the grid is down.

- **Insurers** pay for the insured losses of the full season. Each insurer pays only the part of the season total that is in its layer.

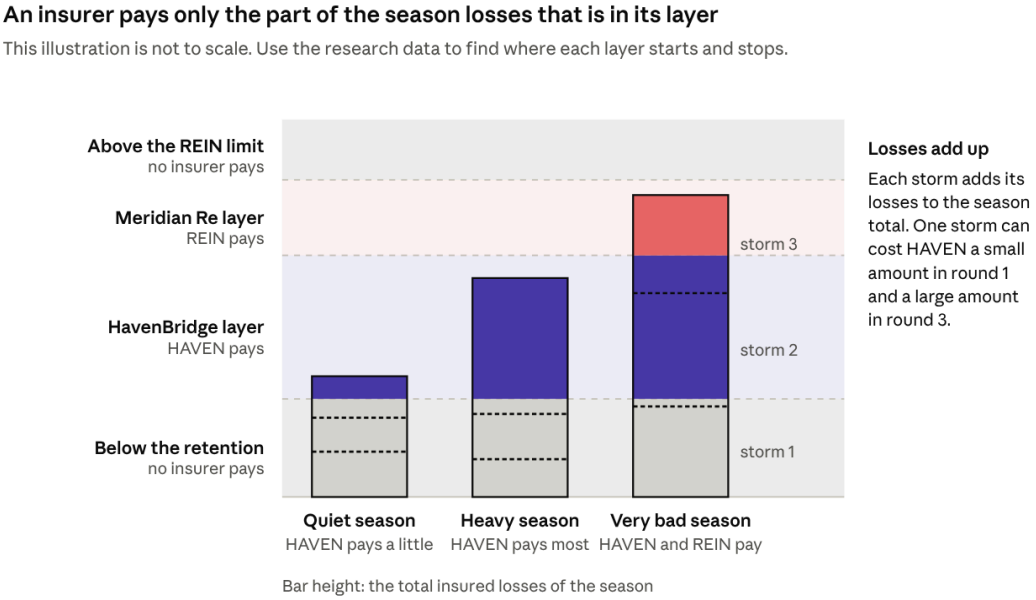


Figure 4. How insurance layers work. Not to scale; the real layer bounds are deliberately not shown.

Because of the layers (Figure 4), HAVEN and REIN can react very differently to the same storm. What an insurer pays depends on where the season total lands, not on the size of any one storm.

From the historical data, you may want to analyze:

- how easily a storm damages each site
- the cost of an outage
- the repair capacity of the rebuilders
- where each insurance layer starts and stops

ETFs

Each ETF trades on its own order book, so its price can drift away from the value of its basket.

When the market is open, you can create or redeem ETF shares:

- **Create:** give the basket and get 1 ETF share.
- **Redeem:** give 1 ETF share and get the basket.

Use the ETF panel or the API: `POST /api/v1/etf/{symbol}/create` and `POST /api/v1/etf/{symbol}/redeem`. Both operations are immediate and have no fee. The exchange books the stocks at their marks and the ETF share at the value of the basket.

You do not need to hold the basket first. A create or redeem can leave you short, as long as you stay within the position limits. Each create or redeem is one order action.

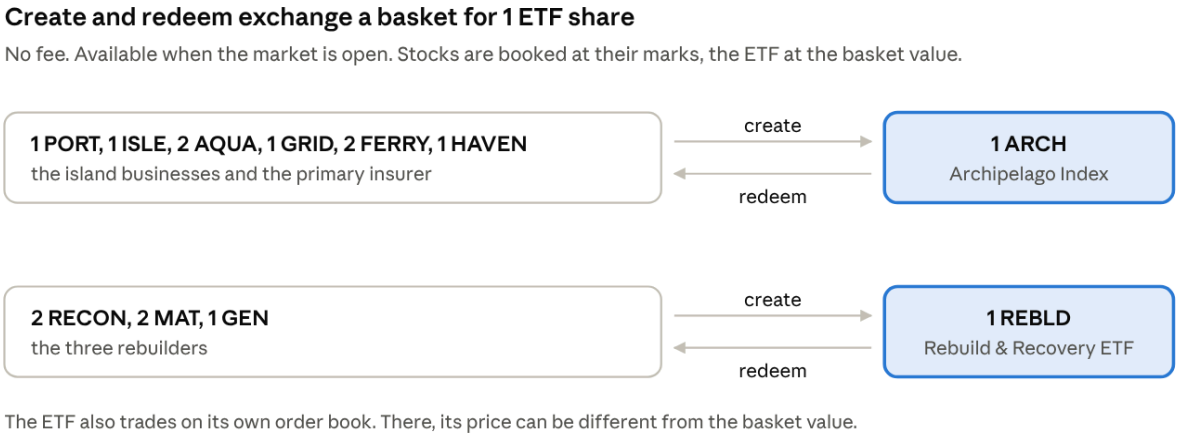


Figure 5. The two ETF baskets, as defined in `case.json`. Create and redeem have no fee and are only available while the market is open.

Settlement

At the close of round 3, after the final damage report:

- each stock settles at its fundamental value
- each ETF settles at the value of its basket

Your positions are marked at these values for scoring (Section 8).

4. What you see

All data about the current condition of a storm is public. The feed sends this data every 2 seconds. The future path of the storm is not public.

The storm

Each storm has a position, a velocity, a radius and an intensity. Wind falls off quickly with distance from the center. The radius sets how far the strong wind reaches; the intensity sets how strong it is.

A site takes damage when it is inside the wind field of a storm. Closer and stronger storms do more damage.

When the wind at a site exceeds a threshold, the site has hurricane conditions and closes. It reopens when the wind drops back below the threshold.

Storms form south of the map and usually move north. Over its last minute, a storm weakens and then dissipates.

Large-scale steering patterns move each storm. Forecasters describe them with six outlooks:

- Direct strike

- Island chain
- Recurve
- Stall
- Eyewall replacement
- Erratic

Erratic covers any storm that does not follow one of the other five patterns. Any storm can turn erratic.

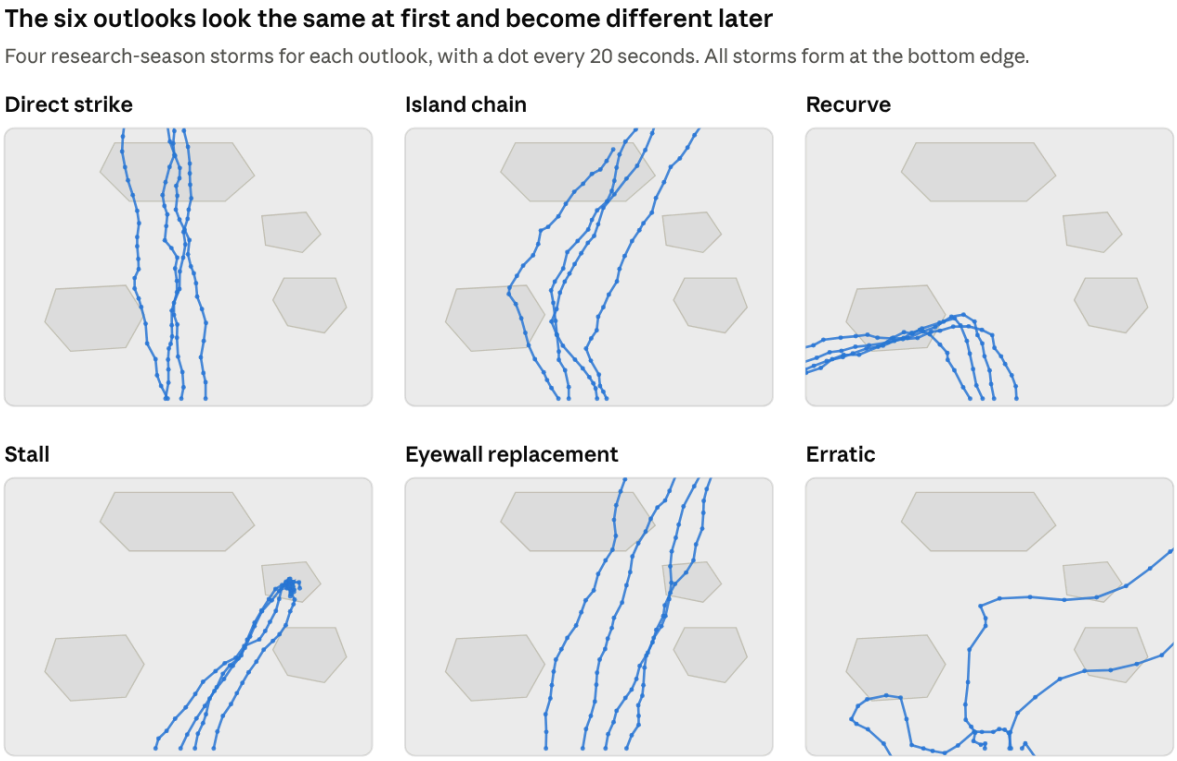


Figure 6. Sample tracks for each outlook: four storms per outlook from the research seasons, plotted with a dot every 20 seconds.

Most storms head north for their first minute (Figure 6), so Direct strike, Island chain and Recurve look alike early on. They separate only when the storm turns.

Sources

Source	Time	Contents
Storm position	Every 2 s	Position, velocity, radius and intensity
Track	Every 20 s	The path of the storm until now
Official advisory	Every 60 s, from 30 s after the storm forms	Outlook, probability of hurricane conditions on each island, forecast cone

Source	Time	Contents
Wire chatter	At random times, approximately every 45 s	A rumored outlook
Impact report	When an island first has hurricane conditions	The name of the island
Damage report	Preliminary report 30 s after the storm ends. Final report 40 s before the close.	Damage % for each company, and insured losses. The preliminary report is an estimate. The final report is exact.
News and announcements	At any time during the season	Headlines. An announcement changes the economics of a company from that time.

Storm position and track are measurements. Official advisories and wire chatter are opinions: advisories are usually right but not always, and wire chatter is less reliable. Use the historical data to measure how accurate a source is.

How you get the data

The feed comes on the market-data WebSocket, `/ws/v1/market-data`, on the news channel. Each frame contains one message. The Storm Watch screen of the trading terminal shows the same feed. Bots connect with a bot key in the `X-API-Key` header.

The static case data is at `GET /cases/hurricane.json`. It contains the map, the islands, the sites, the companies, the ETF baskets, the limits, the outlooks and the timing. The appendix documents every message type and field.

5. The market

The house provides liquidity in every symbol for the whole round, so you can always trade. It quotes around its consensus: its own estimate of each company's value. The consensus is reasonable but imperfect:

- It reacts slowly. After an announcement, it takes about 45 real seconds to adjust fully.
- Its errors show up in the research data, so you can learn them.

Three kinds of participants trade:

- **House market makers** quote bid and ask prices in each symbol around the consensus. They skew their quotes against their inventory and widen their spreads when risk rises.
- **House flow** trades for its own reasons. Some of it is noise, which picks up after news; some of it trades on wire rumors.
- **Other teams** trade on the same order books. Your orders can fill against their orders. Their trades change the prices that you see.

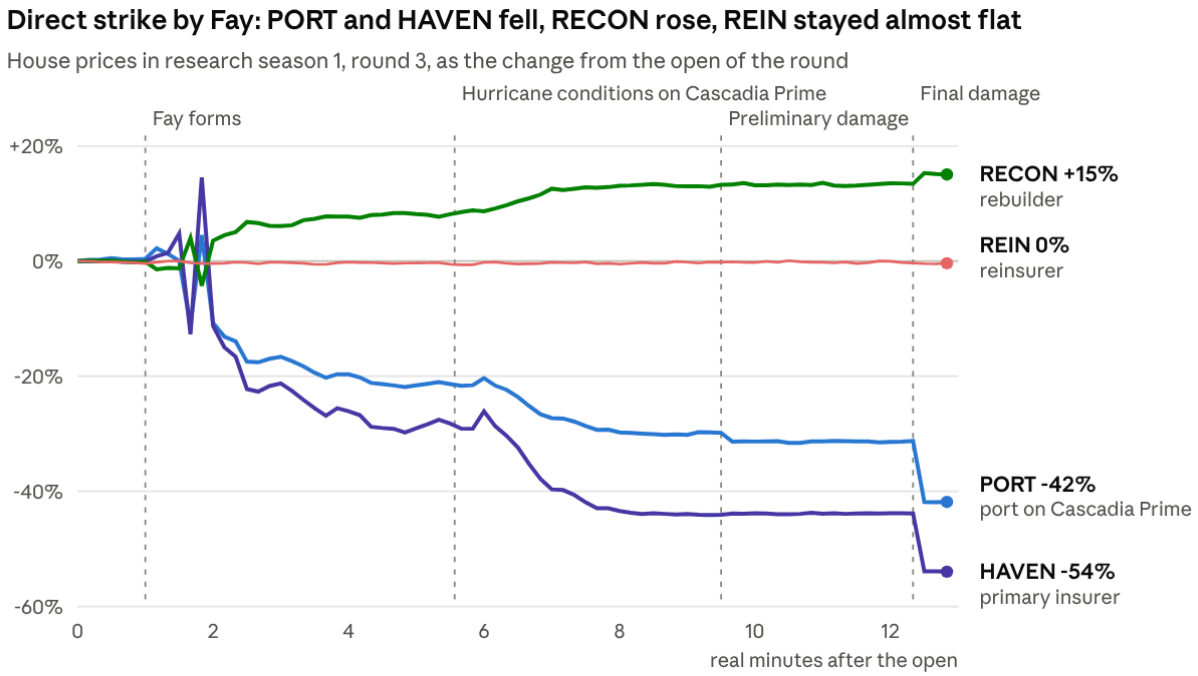


Figure 9. House prices in research season 1, round 3, as the change from the open. Dashed lines mark the storm events. Source: `market.csv`, sampled every 10 s.

Figure 9 shows round 3 of research season 1. After the first advisories for Fay, the house repriced for a direct strike within a minute. The prices changed again at the final damage report. HAVEN fell more than PORT, because HavenBridge paid most of the insured losses of the season. REIN barely moved.

6. Research data and starter kit

You get 600 past seasons, with a total of 1,800 storms. The past seasons and the live season come from the same simulator, with the same map, companies, feed and timing. Use the research data to estimate the numbers that this packet does not give:

- how damage changes the value of each company
- the accuracy of each source
- where each insurance layer starts and stops
- how the house market operates

The dates of season N are approximately N years before the live season. No storm in the research data is a storm of the live season.

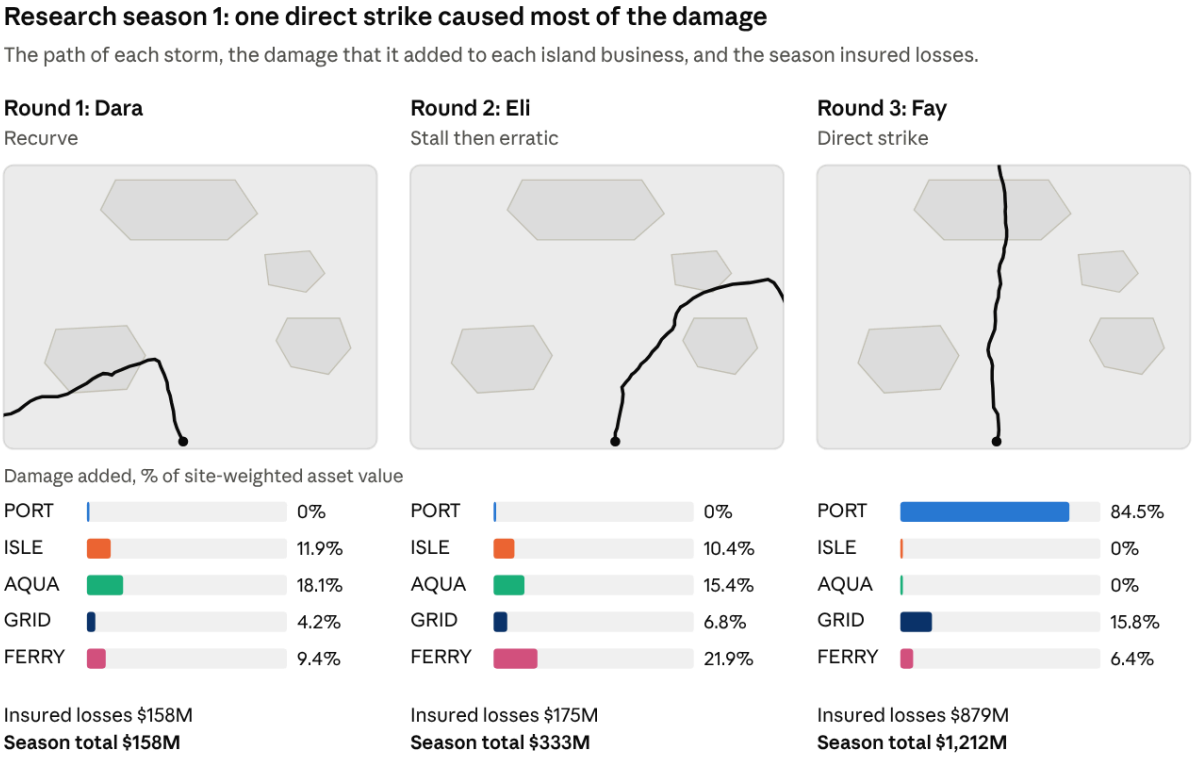


Figure 10. Research season 1, storm by storm: the path of each storm, the damage it added to each island business, and the running total of insured losses. Source: `manifest.json` and `feed.jsonl`.

Figure 10 shows research season 1. In rounds 1 and 2, storms Dara and Eli caused moderate damage on Palm Atoll, Coral Key and Azure Isle. After round 2, the insured losses of the season were $333M. In round 3, storm Fay hit the port directly and added $879M. The manifest gives the true pattern and the damage of each storm. The feed gives all the information that was available during the season.

The download, `hurricane-season-research.zip`, contains these files:

Path	Contents
`case.json`	The public case data: map, islands, sites, companies, ETF baskets, limits, outlooks and timing
`history/manifest.json`	Each season and storm, with the true pattern, the damage and the insured losses
`history/season-NNN/feed.jsonl`	The feed of the season, as the exchange published it
`history/season-NNN/market.csv`	The house prices every 2 s
`history/season-NNN/settlement.json`	The settlement value of each symbol

Path	Contents
`starter_kit/`	A data loader, a feed recorder, a recording check for the testnet and an example bot
`README.md`	The data dictionary for the files. The appendix of this packet describes the feed messages.

The research data does not include:

- News or announcements. The live season can have them.
- The model's parameters. Section 3 lists the ones you need to estimate.
- Order books. Use the testnet to see spreads, depth and the house quotes.

Reference code

The file `history.py` in the starter kit loads the data. It uses only the Python standard library. The example below measures how often the official advisories name a storm's true pattern. Run it from the `starter_kit` folder. Use it as a starting point for your own analysis.

```
from history import load

data = load("../history")
right = total = 0
for season in data.seasons:
    for storm in season.storms:
        flip = storm["turned_erratic_at"]
        for adv in season.messages(round=storm["round"], type="advisory",
source="official"):
            truth = "erratic" if flip is not None and adv["age"] >= flip else
storm["pattern"]
            right += adv["outlook"] == truth
            total += 1
print(f"official advisories named the true pattern {right / total:.0%} of the time")
```

7. The testnet

The test exchange is not ready yet, but we will update once it is ready. It is a copy of the competition exchange. It replays the research seasons, one after the other, until the competition. Use it to confirm that your bot can:

- connect to the exchange
- read each message type
- place and cancel orders
- operate correctly with the market clock

Same as the live case	Different from the live case
The exchange, its rules, order types, limits and rate limits	The storms. The testnet uses the research seasons, in sequence.
Message formats and round timing	Announcements. The research seasons have no announcements.
Tickers, map, companies and ETF baskets	Dates and results
The house market and the price protection	The pauses between rounds

Each season replays exactly as it is in the research data:

- The messages are the same, with the same `seq`.
- The house quotes around the prices of that season.
- The symbols settle at the settlement values of that season.

The market-clock date tells you which season plays. Because the replay is exact, you can verify your feed handling message by message:

1. Record the feed with `record_feed.py`.
2. Compare your recording with the research data. Use `check_recording.py`.
3. Make sure that the check prints “recording matches the published feed”.

When a season ends, the testnet starts the next one:

- Your account, team, bot keys and WebSocket connection stay the same.
- Your positions go back to zero.
- The competition gets a new id.

After each close, and after the testnet prepares a new season, the clock stops for 2 minutes. When the clock starts again, the market opens after 1 real minute.

To start on the testnet:

1. Sign in with Google.
2. Create a team (if you are the first member of your team to log in). Otherwise, join your team.
3. Ask your captain to make a bot key on the Team page.
4. Send the key in the `X-API-Key` header from your bot.
5. Send a `User-Agent` header with each HTTP request. The network in front of the exchange refuses the default `User-Agent` of the Python urllib library.

Before the event

- ☐ Download `hurricane-season-research.zip` and read `README.md`.
- ☐ Load the data with `starter_kit/history.py`. Run the reference code.
- ☐ Build a valuation model for each company. Include the effects of damage, outages and season losses.

- [ ] Measure the accuracy of the official advisories and the wire chatter. Also measure how early each one identifies the true pattern.
- [ ] Sign in on the testnet, join a team and get a bot key from your captain.
- [ ] Connect your bot. Subscribe to news, then place an order and cancel it.
- [ ] Make sure that your bot stays connected when a new season starts.
- [ ] Record a season with `record_feed.py`. Check the recording with `check_recording.py`.
- [ ] Trade a full season on the testnet, by hand and with your bot.

8. Rules and scoring

Trading rules

- **Position limits:** Each symbol has a position limit (Section 3). The limit applies to long and short positions. The limit includes your position and your resting orders on the same side. If an order goes past the limit, the exchange makes the order smaller or rejects it. If the part of an order that fills immediately needs room under the limit, the exchange cancels your resting orders on the same side, worst price first.
- **Orders:** You can send limit orders and market orders, and you can cancel orders. You cannot modify an order; cancel it and send a new one. Prices can have up to 6 decimal places. You can have a maximum of 200 resting orders in each symbol. The exchange prevents self-trades. If your new order crosses your own resting order, the exchange cancels the resting order.
- **Rate limits:**
    - 60 order actions each minute for each bot key, and 180 for each team.
    - 20 orders each minute from the browser.
    - 100 requests each second for each team, for all of the API.
    - 25 market-data connections for each team.
    - An order action is one of these: place an order, cancel an order, cancel all orders, create or redeem ETF shares.

API

All endpoints start with `/api/v1`. Send your bot key in the `X-API-Key` header.

Method and path	Use
`POST /orders`	Place an order. Body: `{"symbol": "PORT", "side": "buy", "order_type": "limit", "quantity": 10, "price": 72.5}`. A market order has no price.
`DELETE /orders/{order_id}`	Cancel one order.
`DELETE /orders/all`	Cancel all your orders. Add `?symbol=PORT` to cancel only the orders in one symbol.

Method and path	Use
`GET /orders/open`	Get your resting orders.
`GET /positions`	Get your positions, with marks and unrealized PnL.
`GET /orderbook/{symbol}`	Get the order book of a symbol.
`GET /competitions/time`	Get the market-clock time and the state of the competition.
`POST /etf/{symbol}/create` and `POST /etf/{symbol}/redeem`	Create or redeem ETF shares. Body: `{"quantity": 1}`.

The file `example_bot.py` in the starter kit shows how to use these endpoints.

Scoring

Your score is the total profit and loss (PnL) of your team after the final settlement at the end of round 3. The score includes:

- your position, marked at its settlement value
- all profit and loss that you realized during the season

The team with the highest PnL wins.

Appendix: feed reference

The feed comes on the market-data WebSocket `/ws/v1/market-data`. Connect with a bot key in the `X-API-Key` header, or with your browser session.

Send one subscription message in the first 10 seconds after you connect, for example `{"action": "subscribe", "symbols": [], "channels": ["news"]}`. To get market data, add symbols and the `quotes`, `trades` or `orderbook` channels to the same message. The exchange ignores all subsequent messages. If a client sends more than 10 frames in one second, the exchange closes the connection.

Each frame contains one message, in the form `{"type": "news", "message": {...}}`. The feed does not replay old messages. A client that connects late gets only the messages that the exchange publishes after it subscribes. A snapshot comes every 30 s. The examples below are from research season 1.

Each message has these fields:

Field	Meaning
`type`	`storm`, `track`, `storm_formed`, `storm_dissipated`, `impact`, `advisory`, `damage_report`, `news`, `announcement` or `snapshot`
`round`	The round, 1 to 3
`storm`	The name of the storm of that round
`time`	The market-clock time (fictional US/Eastern)
`seq`	The position of the message in the feed of the season. The value increases. A snapshot has no `seq`. Gaps are normal. Ignore a message if you already have its `seq`.
`headline`	Text for people. All types have a headline, except storm, track and snapshot.

Map coordinates: the map is 1,000 by 700 units. The origin is at the top left, and y increases to the south. Storms form south of the map, at y = 740, and move north. The age of a storm is in real seconds since it formed.

`storm`: one observation of the live storm. `radius` is the size of the storm. `intensity` is its strength, which decreases in the last minute of the storm. `vx` and `vy` give the velocity in map units each second.

```
{"type": "storm", "round": 1, "storm": "Dara", "time": "2025-08-04T10:00:00", "age": 0,
"x": 480.9, "y": 740.0, "vx": 0.0, "vy": -1.8, "radius": 80.0, "intensity": 1.0, "seq":
1}
```

`track`: the full path of the storm until now, as `[age, x, y, radius]` points every 4 seconds of storm age. Use the track to fill gaps in the observations.

```
{"type": "track", "round": 1, "storm": "Dara", "time": "2025-08-04T10:10:00", "age":
20, "points": [[0, 480.9, 740.0, 80.0], [4, 478.3, 732.7, 79.7], [8, 473.9, 725.8,
78.2], "..."], "seq": 12}
```

`storm_formed` and `storm_dissipated`: the start and the end of the storm.

```
{"type": "storm_formed", "round": 1, "storm": "Dara", "time": "2025-08-04T10:00:00",
"headline": "Dara has formed south of the archipelago.", "seq": 0}
```

`impact`: the first hurricane conditions on an island. The island keys are the same as in `case.json`. Each storm sends a maximum of one impact message for each island.

```
{"type": "impact", "round": 1, "storm": "Dara", "time": "2025-08-04T11:02:00",
"island": "palm_atoll", "age": 124, "headline": "Hurricane conditions reported on Palm
Atoll.", "seq": 76}
```

`advisory` with `source: "official"`: the numbered advisory of the forecast office.

- **`outlook`:** the steering pattern that the forecast office expects. The `outlooks` list in `case.json` gives the keys and names.
- **`hurricane_conditions`:** the probability of hurricane conditions on each island during the remainder of the storm.
- **`cone`:** the forecast track. Each point gives the expected position at `age` seconds into the storm. `spread` is the typical distance of the storm from that point.

```
{"type": "advisory", "round": 1, "storm": "Dara", "time": "2025-08-04T10:15:00",
"source": "official", "number": 1, "age": 30, "outlook": "chain", "outlook_name":
"Island chain", "hurricane_conditions": {"cascadia_prime": 0.78, "coral_key": 0.02,
"azure_isle": 0.0, "palm_atoll": 0.8}, "cone": [{"age": 60, "x": 441.1, "y": 628.6,
"spread": 13.6}, {"age": 90, "x": 419.3, "y": 577.5, "spread": 22.1}, "..."],
"headline": "Advisory 1: steering outlook Island chain. Hurricane conditions: Palm
Atoll 80%, Cascadia Prime 78%.", "seq": 17}
```

`advisory` with `source: "wire"`: a rumor about the outlook of the storm. It uses the same outlook keys. It has no probabilities and no cone.

```
{"type": "advisory", "round": 1, "storm": "Dara", "time": "2025-08-04T10:17:00",
"source": "wire", "age": 34, "outlook": "recurve", "headline": "Wire: forecasters
chatter that Dara will recurve out to sea.", "seq": 21}
```

`damage_report`: the damage that this storm added to each island business, as a percentage of its site-weighted asset value. It also gives the insured losses of this storm in $M. The preliminary report is an early estimate. The final report is the assessed value.

```
{"type": "damage_report", "round": 1, "storm": "Dara", "time": "2025-08-04T14:15:00",
"stage": "preliminary", "damage_pct": {"PORT": 0.0, "ISLE": 9.5, "AQUA": 21.2, "GRID":
6.4, "FERRY": 9.7}, "industry_loss_musd": 166.3, "headline": "Preliminary damage
estimates for Dara: insured losses near $166M.", "seq": 289}
{"type": "damage_report", "round": 1, "storm": "Dara", "time": "2025-08-04T15:40:00",
"stage": "final", "damage_pct": {"PORT": 0.0, "ISLE": 11.92, "AQUA": 18.08, "GRID":
4.23, "FERRY": 9.37}, "industry_loss_musd": 157.5, "headline": "Final damage assessment
for Dara: insured losses $158M.", "seq": 290}
```

`news`: a headline. **`announcement`:** a company or government action that changes the economics of a company from that time. `symbols` lists the companies. The research seasons have no news and no announcements. An `announcement` has this shape:

```
{"type": "announcement", "round": "<round>", "storm": "<storm>", "time": "<market
time>", "headline": "<what changed>", "symbols": ["<ticker>"], "seq": 0}
```

`snapshot`: the current condition of the round. The feed sends a `snapshot` every 30 s, at the open of each round and after a catch-up. A screen or a bot that connects late uses the `snapshot` to get the current condition. The `snapshot` describes the current round, or the round that closed last.

Field	Meaning
`time`	The market-clock time of the snapshot
`phase`	`pre_storm`, `storm`, `aftermath` or `closed`
`last_seq`	The snapshot includes all feed messages up to this `seq`
`obs`	The latest storm message (without type, round and storm), or `null`
`track`	Approximately 40 track points `[age, x, y, radius]` that end at `obs`
`advisory`	The latest official advisory of the round, or `null`
`impacts`	`[{island, age, time}]` for this round
`damage`	All damage reports until now, from all rounds, oldest first
`headlines`	The last 8 headline messages, from all rounds

Delivery: the messages arrive in `seq` order. After a problem on the exchange, the missed messages arrive together and in order. For `storm` and `track`, only the latest message of the group arrives. After a long gap, a snapshot follows. Rarely, a message arrives two times after a network retry. Use `seq` to remove duplicates.

Derivatives Case Packet

Cornell Trading Competition 2026

Disclaimer: All countries, companies, people, and events in this case are fictional. Any resemblance to real countries, companies, people, or events is purely coincidental.

1. Case Overview

The World

There are two countries: Euleria and Newtonia. They are neighbors, and each relies on the other for something it cannot produce on its own.

Euleria sits on rich natural oil reserves and is the world's main supplier of crude oil. Its economy runs on oil production, so anything that disrupts its oil factories or its trade with Newtonia shows up quickly in the price of oil.

Newtonia is home to **NewAir,** which holds a monopoly on air travel. With no competitors, NewAir's stock price depends mostly on two things: how much people trust its planes, and how much it costs to keep them flying. Jet fuel is one of its highest costs, and almost all of that fuel is made from Eulerian oil.

The two nations need each other, but they do not trust each other. They have argued over trade and borders for years, and environmental protest groups in both countries say that oil factories are harming the planet.

Where Things Stand

As the case opens, tensions are rising. Euleria has just elected a new president, Oiler, who is known for making big announcements with little warning. One of his first moves as president was to impose a 20% tariff on trade with Newtonia. NewAir is also under pressure. One of its planes recently crashed, and many people believe the crash was NewAir's fault because the plane was faulty. The official cause has not yet been confirmed.

Your Role

You are a trading team at an options trading firm. You will trade European options on two underlyings: **[OIL]**, the price of crude oil, and **[PLANE]**, the stock price of NewAir. You can post your own markets and trade against other teams and the exchange bots. As news comes out, the prices of both underlyings and their implied volatilities (IV) will change. Your job is to work out how each event affects price and volatility, price the options accordingly, and trade to generate the most profit while managing your risk.

Tradable contracts

Each underlying has 3 European puts and 3 European calls over different strike prices. All contracts share a single expiry, 20 trading days after the case ends. Thus, participants will be trading over 12 contracts total.

You will be able to see the underlying price moving, but you cannot trade it.

How the Case Runs

The case runs over **3 rounds**, and each round represents one trading day in this world. 10 events happen in total: 3 in Round 1, 3 in Round 2, and 4 in Round 3.

Shortly after each event, the news and the new prices of both underlyings are announced publicly. What you learn about volatility depends on the round:

- **Round 1:** the new IV of both underlyings is announced with each event.
- **Round 2:** IV is no longer announced. Instead, 3 analysts (Analyst 1, Analyst 2, Analyst 3) each publish an IV estimate for both underlyings. Reliability varies across analysts and across events, so you will need to combine their estimates with your own market intuition to decide on the IV you trade at.
- **Round 3:** The 3 analysts no longer publish IV estimates. Instead, they will only provide estimates for a quantity linked to IV in some manner. Participants will learn more about this quantity during the live game.

Market Bots

There will be bots on the exchange that provide liquidity and, after a major event, move the market toward the new fair values. **These market bots adjust the underlying price 1 minute after event release, and the correct volatility is priced in for options 2 minutes after event release**. There are also noise bots that trade at random, so not every order in the book reflects new information.

Hint: After an event, there will be a lot of mispricings on the market. Given the event's effects, how can you determine which options are the most mispriced?

2. Calculators

Black-Scholes Calculator

The exchange has a Black-Scholes options pricing calculator built in, so you do not need to write your own. It uses your volatility and underlying price. Other inputs (time to expiry and interest rate) are fixed.

In this competition, we assume that all of the assumptions of Black-Scholes hold. If you wish to learn more about the Black-Scholes model, here is a good place to start.

Implied Volatility Calculator

The exchange also has an IV Calculator tab. Given the market prices, it will tell you the volatility implied by the price. (Essentially, it is the inverse of the Black-Scholes calculator).

Recall that the market bots adjust to the correct volatility 2 minutes after each event. You can use this fact, and your IV calculator, to see which analyst(s) were correct after the market adjusts.

3. Scoring

Within each round, teams are ranked by PnL. Then, the final score is:

$$Final\ Score = \sum_{rounds} (round\ rank)^{1.5}$$

A lower final score is better.

4. Options Basics **(this part is not required; feel free to skip it if you are comfortable with options)**

Definitions

- **Underlying Assets:** the crude oil price [OIL] and the stock price of NewAir [PLANE].
- **European Call Option (Call):** A contract giving the holder the right, but not the obligation, to buy an asset at a predetermined price and on a predetermined date.
- **European Put Option (Put):** A contract giving the holder the right, but not the obligation, to sell an asset at a predetermined price and on a predetermined date.
- **Expiration:** The point at which a derivative contract settles. In this game, all contracts expire 20 trading days after the case ends.
- **Premium (Option Price):** The value of an option (call or put) at the time of trade, determined by the market. In this game, options are traded via bid/ask spreads and executed at the midpoint.
- **Strike Price:** The price at which the option holder can buy (call) or sell (put) the underlying at expiration.
- **Implied Volatility (IV):** The volatility that makes the Black-Scholes price of an option equal to its market price. Higher IV means the market expects bigger price moves, which makes options more expensive.
- **Bid/Ask:** The best prices at which a market participant is willing to buy or sell.
- **PnL (Profit and Loss):** The net financial outcome of all trades made, calculated at the end of each round once the underlying value is revealed.

Brief Options Theory

- **Payoff at Expiration:** At expiration, an option is worth only what it pays out. If S is the underlying price at expiration and K is the strike:
    - Call payoff = max(S − K, 0). The holder buys for K, and the underlying is worth S, so the call only pays when S is above K.
    - Put payoff = max(K − S, 0). The holder sells at K, and the underlying is worth S, so the put only pays when S is below K.

    - Profit for the buyer = payoff − premium paid. The seller's profit is the exact opposite, so every dollar one side makes, the other side loses.
- **Example:** OIL trades at $80. You buy a call with a strike of $85 for a premium of $2.
    - If OIL ends at $90, the payoff is $5, and your profit is $3.
    - If OIL ends at $84, the call expires worthless, and you lose the $2 premium.
    - Your loss as a buyer is capped at the premium. A call seller's loss has no cap.
- **Intrinsic Value and Time Value:** An option's premium has two parts. Intrinsic value is what it would pay if exercised right now, for example max(S − K, 0) for a call. Time value is everything above that. It reflects the chance that the underlying moves favorably before expiration, and it shrinks as expiration approaches.
- **What Drives the Premium:** An option's price depends on the underlying price, the strike, the time to expiration, and volatility. Volatility is the key input in this game. Because an option buyer gains from large favorable moves but loses at most the premium on unfavorable ones, more uncertainty makes **both calls and puts more valuable**.
- **Greeks (Sensitivities):**
    - **Delta:** how much the option price changes for a $1 move in the underlying. Calls have positive delta between 0 and 1. Puts have negative delta between −1 and 0. At-the-money options have a delta of roughly ±0.5.
    - **Vega:** how much the option price changes for a 1-point change in IV. It is positive for both calls and puts and largest for at-the-money options, which makes it the most important Greek for trading on analyst estimates.